GUIA TÉCNICO

SQL: fundamentos e consultas

Filtros, agregações, junções e interpretação dos resultados

Propósito — Material de referência complementar aos notebooks. Exemplos locais com dados sintéticos; laboratórios externos somente sob configuração explícita.

Como utilizar — Leia as explicações, execute os exemplos no ambiente indicado e consulte os notebooks correspondentes.

ROTEIRO DO DOCUMENTO

1. 03 | SQL: fundamentos e consultas
2. Objetivo e escopo
3. Conceito central: Semântica de consulta
4. Aplicação, ressalvas e limites
5. SQL e suas categorias
6. Consultas com filtros e ordenação
7. Join e agregação
8. Exercício resolvido - sensor mais quente

SQL: fundamentos e consultas

Filtros, agregações, junções e interpretação dos resultados

MAPA DE ESTUDO

01 Consultas e predicados SQL

02 Agregação por grupo e indicadores

03 JOIN e cardinalidade

04 Dados ausentes e lógica de três valores

05 Exercícios de consultas auditáveis

Material de referência complementar aos notebooks. Exemplos locais com dados sintéticos; laboratórios externos somente sob configuração explícita.

03 | SQL: fundamentos e consultas

GUIA TÉCNICO APROFUNDADO • ANÁLISE, CÓDIGO E INTERPRETAÇÃO

Objetivo e escopo

Estudo independente baseado no notebook aprovado. Os exemplos locais usam SQLite e dados sintéticos; PostgreSQL, MongoDB e Cassandra requerem serviços de laboratório para verificação real.

Conceito central: Semântica de consulta

A ordem conceitual FROM/JOIN, WHERE, GROUP BY, HAVING, SELECT, ORDER BY ajuda a interpretar agregações. COUNT(*) conta linhas; COUNT(coluna) ignora NULL. Filtre valores do usuário usando parâmetros vinculados, não concatenação.

Aplicação, ressalvas e limites

Evite comparar receitas ou médias sem expor cobertura, unidade de observação e multiplicidade dos JOINs.

SQL e suas categorias

DDL define objetos; DML modifica dados; SELECT recupera linhas. SQL é declarativa: expressamos o resultado, não a ordem imperativa de operações.

Exemplo do notebook — execução local

```
import sqlite3
conn = sqlite3.connect(':memory:')
conn.execute('PRAGMA foreign_keys=ON')
conn.executescript((Path('sql')/'01_schema_sqlite.sql').read_text(encoding='
    utf8'))
leituras = pd.read_csv(DATA/'leituras_sensores.csv')
leituras.to_sql('leituras',conn,if_exists='append',index=False)
print('SQLite inicializado:',conn.execute('SELECT COUNT(*) FROM leituras').f
    etchone()[0], 'leituras')
```

Consultas com filtros e ordenação

A consulta mostra apenas leituras acima de 22°C, com parâmetros vinculados (não concatene valores de usuário em SQL).

Exemplo do notebook — execução local

```
consulta='SELECT id_sensor,data_hora,temperatura_c FROM leituras WHERE tempe
    ratura_c > ? ORDER BY temperatura_c DESC LIMIT 5';display(pd.read_sql_qu
    ery(consulta,conn,params=(22,)))
```

Join e agregação

Agrupar após o JOIN preserva uma linha por cômodo e calcula média de cada grupo. Atenção a duplicações em joins 1:N.

Exemplo do notebook — execução local

```
display(pd.read_sql_query('''SELECT c.nome, COUNT(*) AS n, ROUND(AVG(l.tempe
```

```
ratura_c),2) AS media_c FROM leituras l JOIN sensores s ON s.id_sensor=l
.id_sensor JOIN comodos c ON c.id_comodo=s.id_comodo GROUP BY c.id_comod
o ORDER BY c.nome''',conn))
```

Exercício resolvido - sensor mais quente

Pergunta: qual sensor teve maior média? Solução: agrupar por sensor, ordenar pela média e selecionar primeiro; verificar quantidade para evitar interpretação de uma única observação.

Exemplo do notebook — execução local

```
res=pd.read_sql_query('SELECT id_sensor,COUNT(*) AS n,ROUND(AVG(temperatura_
    c),2) AS media_c FROM leituras GROUP BY id_sensor ORDER BY media_c DESC
    LIMIT 1',conn);display(res);assert res.n.iloc[0]>1
```

Subconsulta e ressalva

Comparar leituras à média global sem misturar médias dos grupos: o denominador é o número de leituras válidas.

Exemplo do notebook — execução local

```
display(pd.read_sql_query('SELECT id_evento,id_sensor,temperatura_c FROM lei
    turas WHERE temperatura_c>(SELECT AVG(temperatura_c) FROM leituras) LIMI
    T 6',conn))
```

--- ### Aprofundamento v2 - NULL, agregação e ordem lógica

Em SQL, NULL representa valor desconhecido ou ausente; comparações como coluna = NULL não funcionam como testes booleanos comuns. Use IS NULL. COUNT(*) conta linhas; COUNT(coluna) ignora valores nulos. Uma consulta sem ORDER BY não garante ordenação.

Interpretação: compare o resultado com a teoria anterior; este exemplo local não substitui um laboratório de serviço externo.

Exemplo do notebook — execução local

```
import sqlite3
with sqlite3.connect(':memory:') as db:
    db.executescript('CREATE TABLE notas(grupo TEXT, nota REAL); INSERT INTO
     notas VALUES ("A",9), ("A",NULL), ("B",7);')
    linhas=db.execute('SELECT grupo, COUNT(*), COUNT(nota), ROUND(AVG(nota),
    2) FROM notas GROUP BY grupo ORDER BY grupo').fetchall()
    print(linhas)
    assert linhas == [('A',2,1,9.0),('B',1,1,7.0)]
```

Exercício de revisão

Exercício resolvido - revisão: identifique a principal hipótese do exemplo acima, explique por que ela importa e cite uma condição na qual a técnica falharia. Resposta orientadora: registre a pré-condição (tipo de dado, chave, restrição, volume ou topologia), teste-a explicitamente e explique os limites dos resultados observados.

Aprofundamento: decisões e estudo de caso

Predicados e semântica de NULL

WHERE aplica predicados antes das agregações. Em SQL, NULL participa de lógica com valor desconhecido: o teste coluna = NULL não é verdadeiro, e IS NULL é o predicado adequado. Filtros que ignoram esse detalhe podem excluir registros silenciosamente. Para preços, temperaturas ou medições, documente se ausentes são erro, falta

de observação ou valor não aplicável.

Junções e multiplicação de linhas

Imagine uma linha por sensor e várias por leitura. JOIN produz uma linha por par correspondente, portanto uma agregação após JOIN pode multiplicar contagens de entidades. COUNT(DISTINCT sensor.id) difere de COUNT(*). Antes de interpretar o resultado, inspecione a granularidade das tabelas, o campo de junção e a quantidade de correspondências.

Agregação auditável

AVG ignora NULL e utiliza apenas valores válidos. Para uma temperatura média por sensor, exponha simultaneamente COUNT(temperatura_c), COUNT(*) e o período observado. Isso impede a comparação acrítica entre um sensor com poucas leituras e outro com dezenas. HAVING filtra grupos agregados, enquanto WHERE filtra linhas antes do agrupamento.

Exercício de consulta

Crie duas consultas equivalentes: uma com GROUP BY e outra usando CTE. Ambas devem listar a temperatura média, a quantidade de registros e o total de medições válidas por cômodo. Compare os resultados e explique a ordem de execução lógica. Teste também uma consulta com placeholder e verifique por que concatenar dados recebidos seria inadequado.

Checklist de interpretação

• Defina a pergunta, unidade de análise e pré-condições da consulta.

• Registre chaves, cardinalidade, restrições e tratamento de valores ausentes.

• Execute os exemplos locais antes de habilitar serviços reais.

• Separe evidência observada de hipóteses e propriedades não testadas.

Leituras e arquivos de apoio

Consulte o notebook correspondente, os scripts em sql/, os datasets sintéticos e a documentação oficial dos SGBDs. Os resultados locais não substituem testes de integração com serviços externos.